

Aula 3 - Manipulação de Arquivos Binários

Teoria e Prática

1. Arquivos binários

Arquivos binários armazenam dados em formato de bytes, não são legíveis diretamente por humanos.

Exemplo de extensão comum: dados.dat, funcionarios.dat

Classes usadas:

- FileOutputStream
- FileInputStream
- DataOutputStream
- DataInputStream
- RandomAccessFile

Definição das classes citadas

Essas classes de Java são usadas para **manipulação de arquivos binários**, mas cada uma tem **funções diferentes**. A diferença principal está em **como os dados são lidos/escritos e no tipo de acesso ao arquivo**.

a) FileOutputStream

O que faz

Escreve **bytes** em um arquivo.

Características

- Escrita **sequencial**
- Trabalha com **dados brutos (bytes)**
- Não entende tipos como int, double, etc.

Exemplo

```
FileOutputStream fos = new FileOutputStream("dados.dat");  
fos.write(65); // escreve o byte 65  
fos.close();
```

Isso grava **apenas um byte** no arquivo.

b) FileInputStream

O que faz

Lê **bytes** de um arquivo.

Características

- Leitura **sequencial**
- Retorna **bytes ou inteiros**
- Não interpreta tipos de dados

Exemplo

```
FileInputStream fis = new FileInputStream("dados.dat");  
int valor = fis.read();  
fis.close();
```

c) DataOutputStream

O que faz

Escreve **tipos primitivos de Java** em formato binário.

Características

- Trabalha sobre outro stream (ex: FileOutputStream)
- Converte tipos em **bytes automaticamente**

Exemplo

```
FileOutputStream fos = new FileOutputStream("dados.dat");  
DataOutputStream dos = new DataOutputStream(fos);
```

```
dos.writeInt(10);  
dos.writeDouble(3.14);
```

```
dos.close();
```

O programador não precisa converter para bytes manualmente.

d) DataInputStream

O que faz

Lê **tipos primitivos** que foram gravados com DataOutputStream.

Características

- Trabalha sobre FileInputStream
- Reconstrói tipos (int, double, etc.)

Exemplo

```
FileInputStream fis = new FileInputStream("dados.dat");  
DataInputStream dis = new DataInputStream(fis);
```

```
int x = dis.readInt();  
double y = dis.readDouble();
```

```
dis.close();
```

⚠ A ordem de leitura deve ser **a mesma da escrita**.

e) RandomAccessFile

O que faz

Permite **acesso aleatório** ao arquivo.

Características

- Pode **ler e escrever**
- Permite usar seek()
- Acessa **qualquer posição do arquivo**

Exemplo

```
RandomAccessFile raf = new RandomAccessFile("dados.dat", "rw");
```

```
raf.seek(8); // vai para o byte 8  
raf.writeInt(20);
```

```
raf.close();
```

Segue exemplo que mostra como gravar arquivo binário

```
import java.io.DataOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class GravarBinario {

    public static void main(String[] args) throws IOException {

        DataOutputStream out = new DataOutputStream(
            new FileOutputStream("dados.dat"));

        out.writeInt(10);
        out.writeDouble(25.5);
        out.writeUTF("Ana");

        out.close();
    }
}
```

Exemplo: ler arquivo binário

```
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class LerBinario {

    public static void main(String[] args) throws IOException {

        DataInputStream in = new DataInputStream(
            new FileInputStream("dados.dat"));

        int numero = in.readInt();
        double valor = in.readDouble();
        String nome = in.readUTF();

        System.out.println(numero);
        System.out.println(valor);
        System.out.println(nome);

        in.close();
    }
}
```

⚠ Importante: A ordem de leitura deve ser a mesma da gravação.

2. Arquivos binários de acesso aleatório (seek)

Quando você precisa acessar uma posição específica no arquivo, use a classe:

- `RandomAccessFile`

Ele permite:

- ler
- gravar
- mover o ponteiro do arquivo

Usando o método:

seek(posição)

Exemplo simples de seek

Cada int ocupa 4 bytes:

10	bytes 0	3
20	bytes 4	7
30	bytes 8	11

Então: seek(4) vai para o segundo número.

Diferença entre os tipos de arquivo

Tipo	Característica
Texto	Legível por humanos
Binário	Armazena bytes
Binário estruturado	Dados organizados em registros

Arquivo binário é diferente de estruturado. Segue a diferença entre eles.

I. Arquivo binário

Um **arquivo binário** é simplesmente um arquivo que armazena **dados em forma de bytes**, que **não são necessariamente legíveis por humanos**.

- Ele não precisa ter uma estrutura fixa ou organizada.

Características

- Sequência de bytes
- Pode conter qualquer tipo de dado

- A interpretação depende do programa

Exemplos

- Imagens (.png, .jpg)
- Áudio (.mp3, .wav)
- PDF
- Executáveis (.exe)

Exemplo conceitual

10110100 01001010 11100010 00011101

O sistema vê apenas **bytes**.

II. Arquivo binário estruturado

Características:

- os dados são **binários**
- estão organizados em **registros**
- cada registro possui **campos definidos**
- os registros têm **tamanho fixo**

Isso permite:

- acesso direto
- atualização no meio do arquivo
- uso de índices

Apenas registros de arquivos binários estruturados podem ser acessados diretamente.

RandomAccessFile é a biblioteca que permite acesso direto usando seek()

Fluxo básico de manipulação de arquivos

- 1 Abrir arquivo
- 2 Ler ou escrever dados
- 3 Fechar arquivo

Exemplo lógico:

abrir → processar → fechar

Exemplo de estrutura de um registro com três campos e respectivos tipos

Campo	Tipo	Bytes
código	int	4
nome	String (30 chars)	60
salário	double	8

Tamanho total do registro

$$4 + 60 + 8 = 72 \text{ bytes}$$

O nome usa **30 caracteres fixos** e cada char ocupa **2 bytes** em Java.

Exemplo de cadastro e leitura de um registro específico

Estrutura do registro

Campo	Tipo	Tamanho
id	int	4 bytes
idade	int	4 bytes
salário	double	8 bytes
nome	String fixa	20 caracteres

Observação: em `RandomAccessFile.writeChar()`, cada char ocupa **2 bytes**.
Logo, nome = 20 chars × 2 bytes = **40 bytes**.

Tamanho total do registro:

$$4 \text{ (id)} + 4 \text{ (idade)} + 8 \text{ (salário)} + 40 \text{ (nome)} = \mathbf{56 \text{ bytes}}$$

Como o arquivo fica organizado (conceitualmente)

Registro 0 → bytes 0 – 55

Registro 1 → bytes 56 – 111

Registro 2 → bytes 112 – 167

Visualização:

```
[ID][IDADE][SALÁRIO][NOME.....]  
[ID][IDADE][SALÁRIO][NOME.....]  
[ID][IDADE][SALÁRIO][NOME.....]
```

Exemplo de acesso direto

Para acessar o **registro 2**:

posição = 2 × 56

posição = 112

Então: raf.seek(112);

Resumo

Java trabalha com arquivos usando streams de entrada e saída. Os principais tipos são:

- Arquivo texto
 - Arquivo binário
 - Arquivo com acesso aleatório (seek)
-

Exemplo de programa CRUD usando gravação e leitura de dados:

O programa terá um menu:

1. Cadastrar funcionário
2. Consultar funcionário pelo código
3. Listar todos
4. Sair

A posição do funcionário no arquivo será:

posição = (codigo - 1) * tamanhoRegistro

seek(posição) – move para o byte da posição indicada.

Programa completo

```
import java.io.IOException;
```

```
import java.io.RandomAccessFile;
```

```
import java.util.Scanner;
```

```
public class CadastroFuncionarios {
```

```
    static final int TAM_NOME = 30;
```

```
    static final int TAM_REGISTRO = 72;
```

```
    public static void main(String[] args) throws IOException {
```

```
        RandomAccessFile arq = new RandomAccessFile("funcionarios.dat", "rw");
```

```
        Scanner sc = new Scanner(System.in);
```



```
int opcao;

do {

    System.out.println("\n1 - Cadastrar");
    System.out.println("2 - Consultar");
    System.out.println("3 - Listar");
    System.out.println("4 - Sair");
    opcao = sc.nextInt();

    switch (opcao) {

        case 1:
            cadastrar(arq, sc);
            break;

        case 2:
            consultar(arq, sc);
            break;

        case 3:
            listar(arq);
            break;
    }

} while (opcao != 4);

arq.close();
}

static void cadastrar(RandomAccessFile arq, Scanner sc) throws IOException {

    System.out.print("Codigo: ");
    int codigo = sc.nextInt();

    sc.nextLine();

    System.out.print("Nome: ");
    String nome = sc.nextLine();

    System.out.print("Salario: ");
    double salario = sc.nextDouble();

    long posicao = (codigo - 1) * TAM_REGISTRO;

    arq.seek(posicao);
```

```
        arq.writeInt(codigo);

        StringBuffer buffer = new StringBuffer(nome);
        buffer.setLength(TAM_NOME);

        arq.writeChars(buffer.toString());

        arq.writeDouble(salario);

        System.out.println("Funcionario gravado.");
    }

    static void consultar(RandomAccessFile arq, Scanner sc) throws IOException {

        System.out.print("Codigo: ");
        int codigo = sc.nextInt();

        long posicao = (codigo - 1) * TAM_REGISTRO;

        if (posicao >= arq.length()) {
            System.out.println("Funcionario inexistente.");
            return;
        }

        arq.seek(posicao);

        int cod = arq.readInt();

        char nome[] = new char[TAM_NOME];

        for (int i = 0; i < TAM_NOME; i++) {
            nome[i] = arq.readChar();
        }

        String nomeStr = new String(nome).trim();

        double salario = arq.readDouble();

        System.out.println("Codigo: " + cod);
        System.out.println("Nome: " + nomeStr);
        System.out.println("Salario: " + salario);
    }

    static void listar(RandomAccessFile arq) throws IOException {

        arq.seek(0);
```

```
while (arq.getFilePointer() < arq.length()) {  
  
    int cod = arq.readInt();  
  
    char nome[] = new char[TAM_NOME];  
  
    for (int i = 0; i < TAM_NOME; i++) {  
        nome[i] = arq.readChar();  
    }  
  
    String nomeStr = new String(nome).trim();  
  
    double salario = arq.readDouble();  
  
    System.out.println(cod + " - " + nomeStr + " - " + salario);  
}  
}  
}
```

Como o seek() funciona neste programa

Exemplo de funcionários gravados:

Código	Posição no arquivo
1	0
2	72
3	144
4	216

Cálculo:

$posicao = (codigo - 1) * 72$

Exemplo:

$codigo = 3$

$posicao = (3 - 1) * 72$

$posicao = 144$

Então: seek(144)

Vai diretamente para o funcionário 3.

Vantagem desse método

- ✓ acesso direto ao registro
- ✓ não precisa ler o arquivo inteiro
- ✓ funciona como tabela de banco simples

Limitações

- △ tamanho do registro precisa ser fixo
- △ alteração de estrutura quebra o arquivo
- △ não possui consultas complexas como banco de dados

Modificações para melhor entendimento:

a) Alterar salário usando seek()

Para alterar apenas o salário não precisamos ler todo o registro, basta ir direto na posição do salário.

Estrutura do registro:

codigo (4 bytes)
nome (60 bytes)
salario (8 bytes)

Logo a posição do salário dentro do registro é:

$$4 + 60 = 64$$

Fórmula da posição $posicao = (codigo - 1) * TAM_REGISTRO + 64$

Método para alterar salário

```
static void alterarSalario(RandomAccessFile arq, Scanner sc) throws IOException {
```

```
    System.out.print("Codigo do funcionario: ");
    int codigo = sc.nextInt();
```

```
    System.out.print("Novo salario: ");
    double salario = sc.nextDouble();
```

```
    long posicao = (codigo - 1) * TAM_REGISTRO + 64;
```

```
    if (posicao >= arq.length()) {
        System.out.println("Funcionario nao encontrado.");
        return;
    }
```

```
    arq.seek(posicao);
    arq.writeDouble(salario);
```

```
    System.out.println("Salario alterado.");
}
```

O que acontece - Exemplo registro 3:

posicao inicial = 144

salario = 144 + 64

salario = 208

Então: seek(208) - Atualiza apenas o salário.

b) Como excluir funcionário em arquivo binário

Em arquivos binários os registros não são removidos fisicamente, pois isso deslocaria todos os outros registros. A solução usada é exclusão lógica.

Ideia: codigo = -1

Isso indica que o registro foi removido.

Método excluir

```
static void excluir(RandomAccessFile arq, Scanner sc) throws IOException {
```

```
    System.out.print("Codigo do funcionario: ");  
    int codigo = sc.nextInt();
```

```
    long posicao = (codigo - 1) * TAM_REGISTRO;
```

```
    if (posicao >= arq.length()) {  
        System.out.println("Funcionario inexistente.");  
        return;  
    }
```

```
    arq.seek(posicao);
```

```
    arq.writeInt(-1);
```

```
    System.out.println("Funcionario excluido.");  
}
```

Listagem ignorando excluídos

```
if(cod != -1){  
    System.out.println(cod + " - " + nome + " - " + salario);  
}
```

c) Criar índice para busca mais rápida

Quando o arquivo é grande, buscar funcionário percorrendo todo o arquivo é lento.

Solução: arquivo de índice.

1. Como o índice fica no arquivo

Arquivo indice.dat:

cod	posição
101	0
102	16
103	32
104	48

Cada linha significa: código → posição no arquivo de dados

2. Buscar o índice procurado

1. Ler o índice
2. Encontrar o código
3. Usar seek() na posição

Exemplo:

`dados.seek(posicao);` E então ler o registro.

3. Fluxo completo do sistema

`funcionarios.dat` → arquivo de dados

`indice.dat` → arquivo de índice

Busca funciona assim:

usuario busca codigo



procura no índice



descobre posição



`seek(posicao)`



lê registro

4. Vantagem do índice

Sem índice	Com índice
Busca sequencial	Busca direta
Mais lento	Muito mais rápido
Lê vários registros	Lê apenas 1

Resumo

Um **arquivo de índice** guarda: chave → posição do registro

A busca pode ser sequencial ou direta se usar índice.

Classe de índice

```
class Indice {  
    int codigo;  
    long posicao;  
  
}
```

Criar índice

```
static void criarIndice(RandomAccessFile arq) throws IOException {  
  
    RandomAccessFile indice = new RandomAccessFile("indice.dat", "rw");  
  
    arq.seek(0);  
  
    while (arq.getFilePointer() < arq.length()) {  
  
        long pos = arq.getFilePointer();  
  
        int cod = arq.readInt();  
  
        char nome[] = new char[30];  
  
        for(int i=0;i<30;i++)  
            nome[i] = arq.readChar();  
  
        double salario = arq.readDouble();  
  
        if(cod != -1){  
            indice.writeInt(cod);  
            indice.writeLong(pos);  
        }  
    }  
  
    indice.close();  
}
```

Buscar usando índice

```
static void buscarIndice(RandomAccessFile arq, int codigo) throws IOException {  
  
    RandomAccessFile indice = new RandomAccessFile("indice.dat", "r");  
  
    while(indice.getFilePointer() < indice.length()){  
  
        int cod = indice.readInt();  
        long pos = indice.readLong();  
  
        if(cod == codigo){  
  
            arq.seek(pos);  
  
            int c = arq.readInt();  
  
            char nome[] = new char[30];  
  
            for(int i=0; i<30; i++)  
                nome[i] = arq.readChar();  
  
            double sal = arq.readDouble();  
  
            System.out.println(c+" "+new String(nome).trim()+" "+sal);  
  
            break;  
        }  
    }  
  
    indice.close();  
}
```

Comparação de desempenho

Método	Complexidade
Busca sequencial	$O(n)$
Busca com índice	$O(\log n)$ ou $O(1)$ dependendo da estrutura

Exemplo de arquivo de índice ordenado – pesquisa usando índice

Suponha o arquivo indice.dat:

Registro	Código	Posição
0	100	0
1	120	32
2	140	64
3	160	96
4	180	128
5	200	160
6	220	192

Queremos encontrar o **código 180**.

I. Passo 1 — pegar o registro do meio

Número de registros = 7

início = 0

fim = 6

meio = $(0 + 6) / 2 = 3$

Registro 3 – **código = 160**

Comparação: $180 > 160$, então o valor está **à direita**.

Novo intervalo:

Início = 4

fim = 6

II. Passo 2 — calcular novo valor do meio

meio = $(4 + 6) / 2 = 5$

Registro 5: **Código = 200**

Comparação: $180 < 200$

Então o valor está **à esquerda**.

Novo intervalo:

início = 4

fim = 4

III. Passo 3 — novo meio

meio = $(4 + 4) / 2 = 4$

Registro 4: **Código = 180**

Encontrado!

Posição no arquivo de dados: 128

Basta usar esta posição no Java para encontrar o registro procurado

```
dados.seek(128);
```

IV. Visualização da busca

[100] [120] [140] [160] [180] [200] [220]

↑
meio

180 é maior que 160:

[180] [200] [220]

↑
início

Depois:

[180] [200]

↑

Encontrado.

V. Quantidade de comparações

Registros	Busca sequencial	Busca binária
100	até 100	~7
1000	até 1000	~10
1 milhão	até 1 milhão	~20

Por isso índices usam **busca binária**, é muito mais rápido.

Exemplo prático para fixação!

Sistema em Java para cadastro de produtos utilizando arquivo binário com acesso direto.

Cada produto possui:

Campo	Tipo	Tamanho
Código	int	4 bytes
nome	char[30]	60 bytes
preço	double	8 bytes

Tamanho do registro: 72 bytes

O sistema deve permitir:

- Cadastrar produto
- Consultar produto pelo código
- Alterar preço
- Excluir produto
- Listar produtos
- Criar índice
- Buscar usando índice

Estrutura do registro

codigo	4 bytes
nome	60 bytes
preco	8 bytes

TOTAL = 72

Constantes usadas:

```
static final int TAM_NOME = 30;
```

```
static final int TAM_REG = 72;
```

Programa completo

```
import java.io.RandomAccessFile;  
import java.io.IOException;  
import java.util.Scanner;
```

```
public class CadastroProdutos {
```

```
    static final int TAM_NOME = 30;  
    static final int TAM_REG = 72;
```

```
    public static void main(String[] args) throws Exception {
```

```
        RandomAccessFile arq = new RandomAccessFile("produtos.dat", "rw");
```

```
Scanner sc = new Scanner(System.in);

int op;

do {

    System.out.println("\n1 Cadastrar");
    System.out.println("2 Consultar");
    System.out.println("3 Alterar preco");
    System.out.println("4 Excluir");
    System.out.println("5 Listar");
    System.out.println("6 Criar indice");
    System.out.println("7 Buscar indice");
    System.out.println("8 Sair");

    op = sc.nextInt();

    switch (op) {

        case 1: cadastrar(arq, sc); break;
        case 2: consultar(arq, sc); break;
        case 3: alterarPreco(arq, sc); break;
        case 4: excluir(arq, sc); break;
        case 5: listar(arq); break;
        case 6: criarIndice(arq); break;
        case 7: buscarIndice(arq, sc); break;

    }

} while (op != 8);

arq.close();
}

static void cadastrar(RandomAccessFile arq, Scanner sc) throws IOException {

    System.out.print("Codigo: ");
    int cod = sc.nextInt();
    sc.nextLine();

    System.out.print("Nome: ");
    String nome = sc.nextLine();

    System.out.print("Preco: ");
    double preco = sc.nextDouble();

    long pos = (cod - 1) * TAM_REG;

    arq.seek(pos);

    arq.writeInt(cod);

    StringBuffer buffer = new StringBuffer(nome);
    buffer.setLength(TAM_NOME);
```

```
        arq.writeChars(buffer.toString());

        arq.writeDouble(preco);

        System.out.println("Produto gravado.");
    }

    static void consultar(RandomAccessFile arq, Scanner sc) throws IOException {

        System.out.print("Codigo: ");
        int cod = sc.nextInt();

        long pos = (cod - 1) * TAM_REG;

        if (pos >= arq.length()) {
            System.out.println("Produto inexistente.");
            return;
        }

        arq.seek(pos);

        int codigo = arq.readInt();

        char nome[] = new char[TAM_NOME];

        for (int i = 0; i < TAM_NOME; i++)
            nome[i] = arq.readChar();

        double preco = arq.readDouble();

        if (codigo != -1)
            System.out.println(codigo + " " + new String(nome).trim() + " " + preco);
    }

    static void alterarPreco(RandomAccessFile arq, Scanner sc) throws IOException {

        System.out.print("Codigo: ");
        int cod = sc.nextInt();

        System.out.print("Novo preco: ");
        double preco = sc.nextDouble();

        long pos = (cod - 1) * TAM_REG + 64;

        if (pos >= arq.length()) {
            System.out.println("Produto inexistente.");
            return;
        }

        arq.seek(pos);

        arq.writeDouble(preco);

        System.out.println("Preco alterado.");
    }
```

```
static void excluir(RandomAccessFile arq, Scanner sc) throws IOException {

    System.out.print("Codigo: ");
    int cod = sc.nextInt();

    long pos = (cod - 1) * TAM_REG;

    if (pos >= arq.length()) {
        System.out.println("Produto inexistente.");
        return;
    }

    arq.seek(pos);

    arq.writeInt(-1);

    System.out.println("Produto excluido.");
}
```

```
static void listar(RandomAccessFile arq) throws IOException {

    arq.seek(0);

    while (arq.getFilePointer() < arq.length()) {

        int cod = arq.readInt();

        char nome[] = new char[TAM_NOME];

        for (int i = 0; i < TAM_NOME; i++)
            nome[i] = arq.readChar();

        double preco = arq.readDouble();

        if (cod != -1)
            System.out.println(cod + " " + new String(nome).trim() + " " + preco);
    }
}
```

```
static void criarIndice(RandomAccessFile arq) throws IOException {

    RandomAccessFile indice = new RandomAccessFile("indice.dat", "rw");

    arq.seek(0);

    while (arq.getFilePointer() < arq.length()) {

        long pos = arq.getFilePointer();

        int cod = arq.readInt();

        char nome[] = new char[TAM_NOME];
```

```
        for (int i = 0; i < TAM_NOME; i++)
            nome[i] = arq.readChar();

        double preco = arq.readDouble();

        if (cod != -1) {
            indice.writeInt(cod);
            indice.writeLong(pos);
        }
    }

    indice.close();

    System.out.println("Indice criado.");
}

static void buscarIndice(RandomAccessFile arq, Scanner sc) throws IOException {

    System.out.print("Codigo: ");
    int codigo = sc.nextInt();

    RandomAccessFile indice = new RandomAccessFile("indice.dat", "r");

    while (indice.getFilePointer() < indice.length()) {

        int cod = indice.readInt();
        long pos = indice.readLong();

        if (cod == codigo) {

            arq.seek(pos);

            int c = arq.readInt();

            char nome[] = new char[TAM_NOME];

            for (int i = 0; i < TAM_NOME; i++)
                nome[i] = arq.readChar();

            double preco = arq.readDouble();

            System.out.println(c + " " + new String(nome).trim() + " " + preco);
            break;
        }
    }

    indice.close();
}
}
```